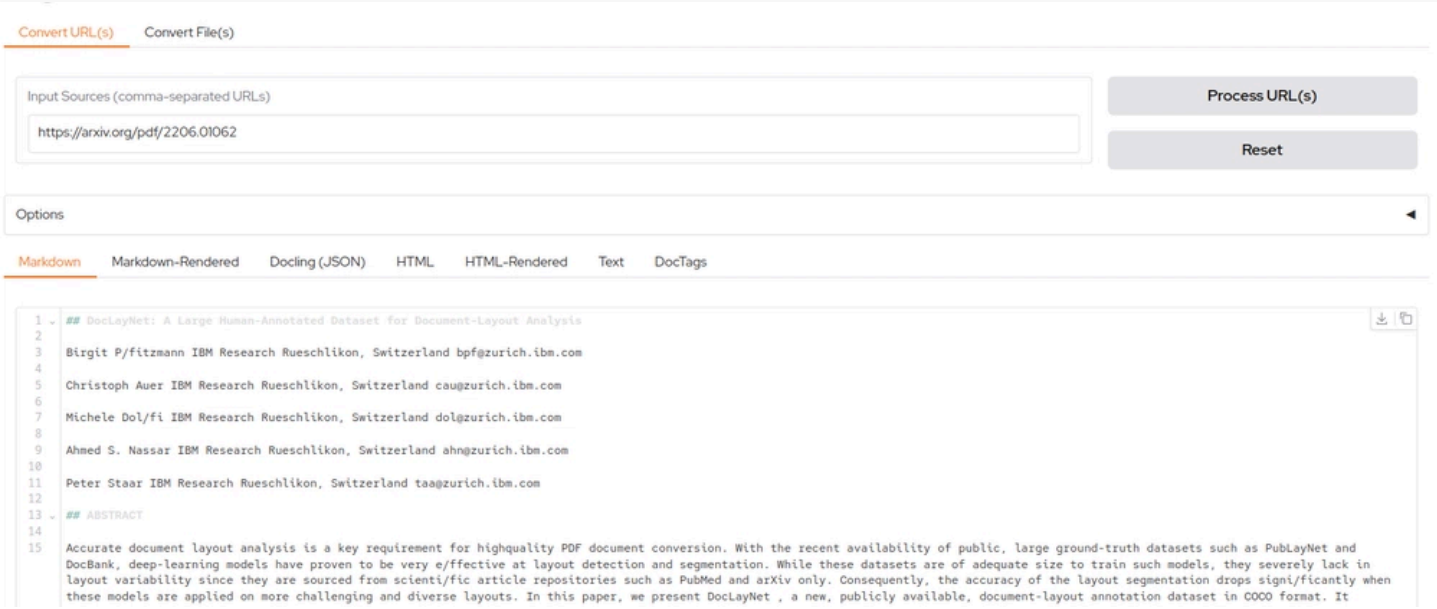


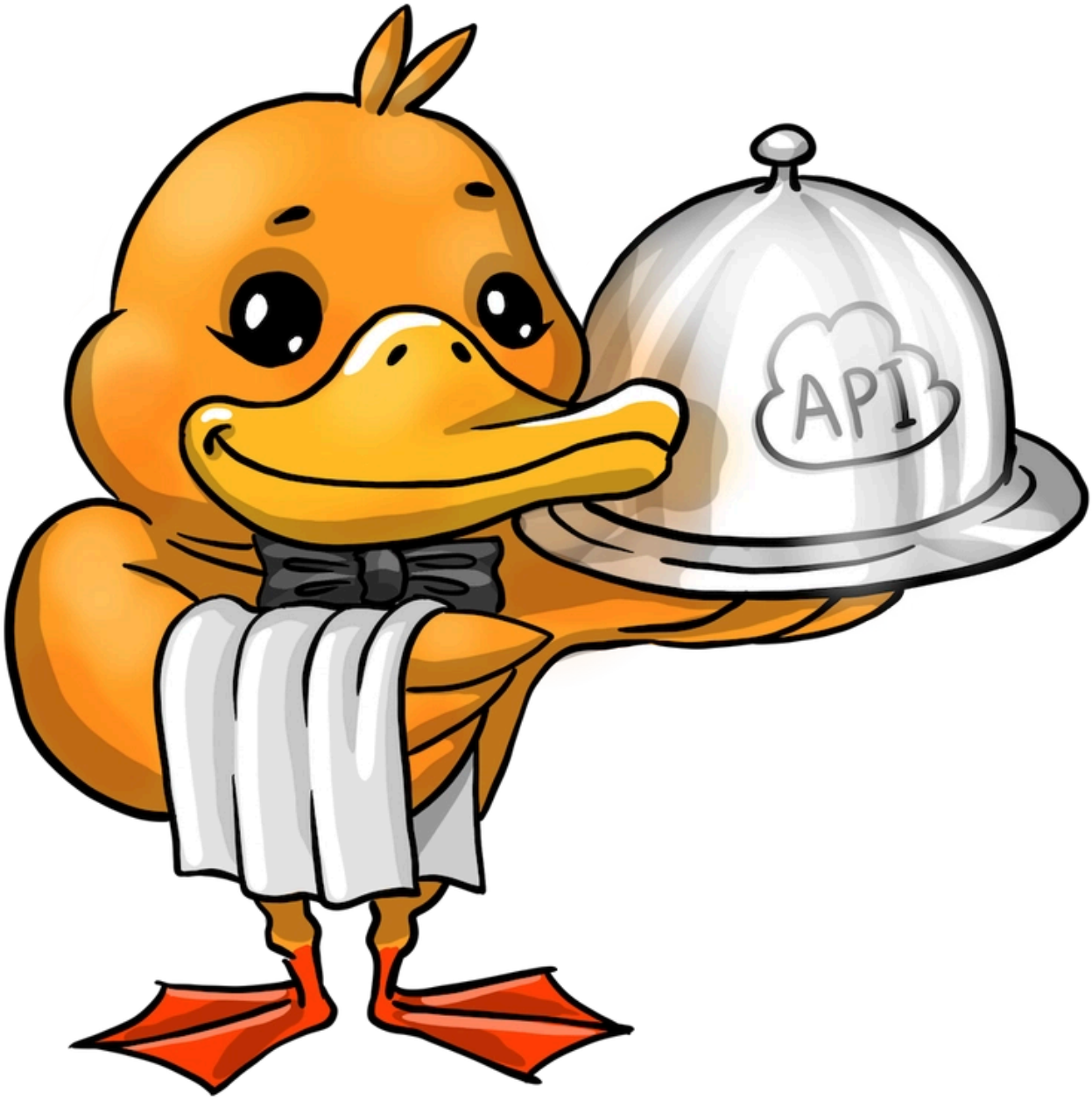
Alain Airom
Posted on 17 Apr 2025 • Edited on 29 Aug 2025

5

Running Docling as an API Server

#docling #rag #llm #api

How to start with “docling serve”!



Introduction

Docling offers a user-friendly interface to leverage its capabilities as an API server. While its API and comprehensive documentation undoubtedly empower the development of custom applications, Docling also features an integrated API server ready for deployment in diverse environments, from on-premise infrastructure to cloud and Kubernetes-based platforms. Let’s delve deeper into its potential.

Local Implementation with a container image

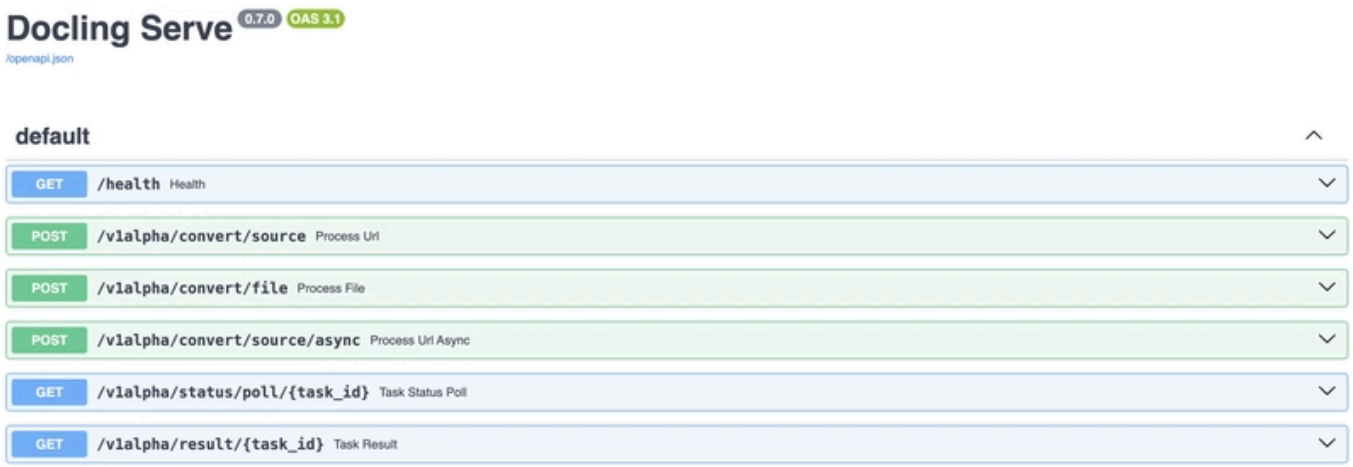
The easiest way to test the API server locally is to use Podman/Docker. To do this, just write down the following command in a terminal on your machine (assuming you have either Podman or Docker installed locally).

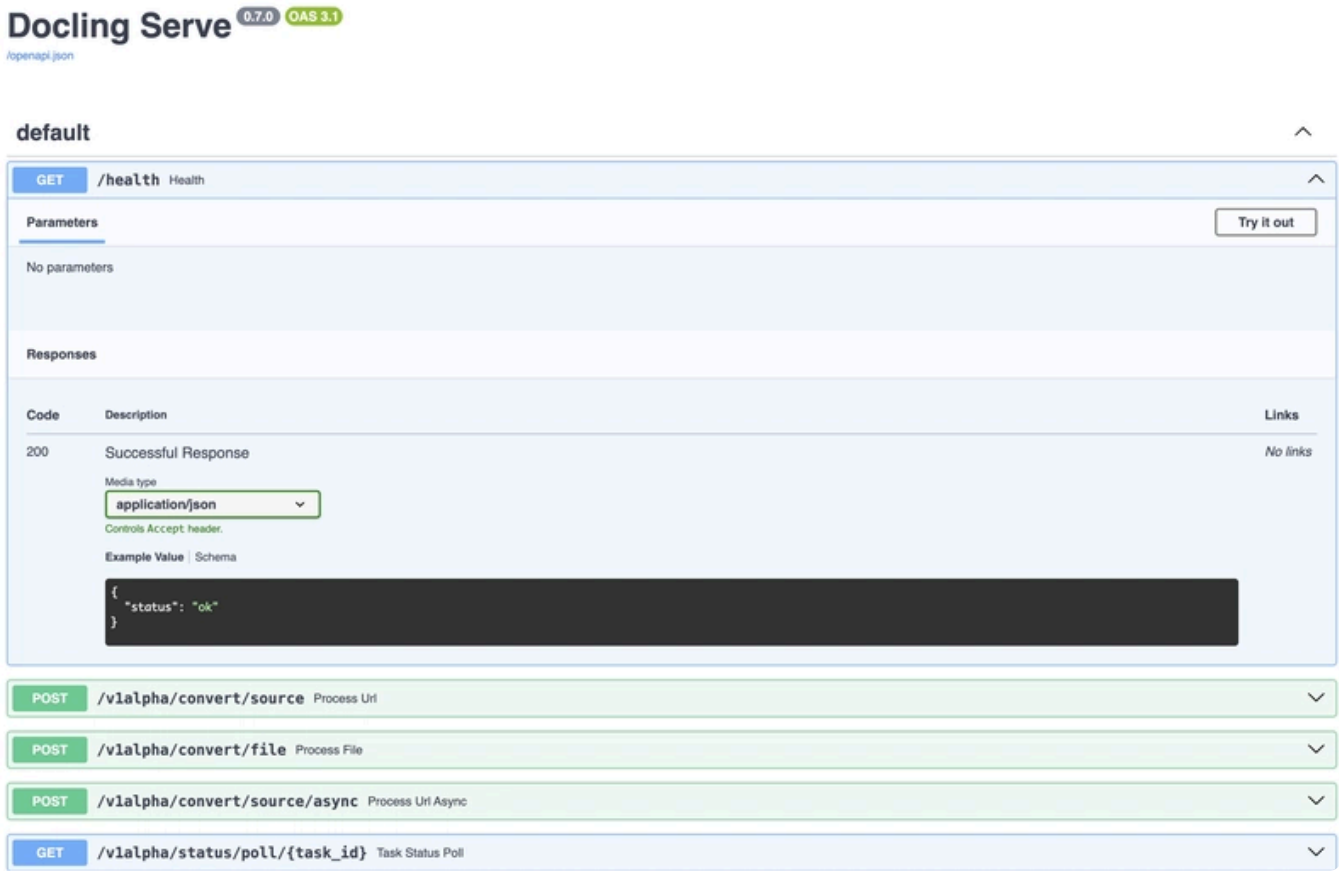
```
# Using container images, e.g. with Podman
podman run -p 5001:5001 quay.io/docling-project/docling-serve
# or
docker run -p 5001:5001 quay.io/docling-project/docling-serve
```

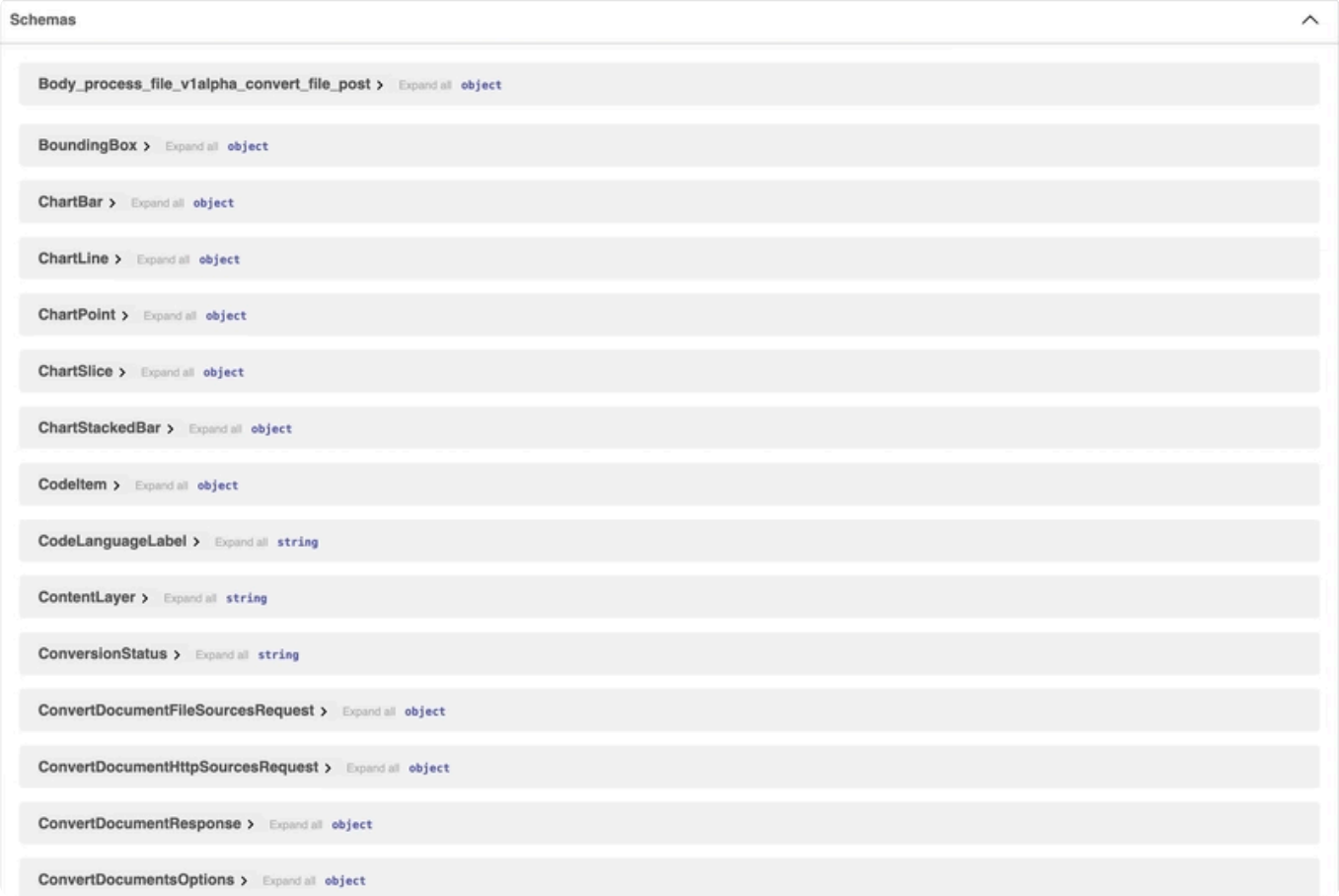
You will have three URLs to access the local server.

```
Server started at http://0.0.0.0:5001
Documentation at http://0.0.0.0:5001/docs
UI at http://0.0.0.0:5001/ui
```

You have access to the interface using the "http://0.0.0.0:5001/docs".







You can already test with a CURL command.

```
curl -X 'POST' \
  'http://localhost:5001/v1alpha/convert/source' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "http_sources": [{"url": "https://arxiv.org/pdf/2501.17887"}]
  }'
```

Local Implementation with code

I also tested the service by writing a very simple Python code which is provided below.

- Environment preparation.

```
python3.12 -m venv myenv
source myenv/bin/activate

# Using the python package
pip install --upgrade pip
pip install "docling-serve"
```

```
docling-serve run
```

- Sample code.

```
import httpx
import asyncio
import json
import os

async def convert_from_url(url_to_convert):
    """
    Asynchronously sends a URL to a local server for conversion.

    Args:
        url_to_convert (str): The URL of the resource to be converted.

    Returns:
        dict or None: The JSON response from the server if successful, None c
    """
    async with httpx.AsyncClient(timeout=60.0) as async_client:
        api_url = "http://localhost:5001/v1alpha/convert/source"
        headers = {
            'accept': 'application/json',
            'Content-Type': 'application/json'
        }
        data = {
            "http_sources": [{"url": url_to_convert}]
        }

        try:
            response = await async_client.post(api_url, headers=headers, jsor
            response.raise_for_status()  # Raise an exception for bad status
            return response.json()
        except httpx.HTTPError as e:
            print(f"HTTP error occurred: {e}")
            return None
        except json.JSONDecodeError as e:
            print(f"Error decoding JSON response: {e}")
            print(f"Raw response: {response.text}")
            return None
        except Exception as e:
            print(f"An unexpected error occurred: {e}")
            return None

def write_to_file(file_path, data):
    """
```

```
    Writes the given data to a file.

    Args:
        file_path (str): The path to the file to write to.
        data (dict): The data to write (will be written as JSON).
    """
    try:
        with open(file_path, 'w') as f:
            json.dump(data, f, indent=4)  # Use indent for pretty printing
        print(f"Successfully wrote data to: {file_path}")
    except Exception as e:
        print(f"Error writing to file {file_path}: {e}")

async def main():
    """
    Main function to run the URL-based file conversion and write the output t
    """
    target_url = "https://arxiv.org/pdf/2501.17887"
    output_file = "conversion_output.json"  # You can change the filename her

    result = await convert_from_url(target_url)

    if result:
        print("URL conversion successful!")
        write_to_file(output_file, result)
    else:
        print("URL conversion failed. No output to write.")

if __name__ == "__main__":
    asyncio.run(main())
```

Running the provided code as-is will provide the following JSON format output (excerpt of the output, as per my implementation).

```
{
    "document": {
        "filename": "2501.17887v1.pdf",
        "md_content": "## Docling: An Efficient Open-Source Toolkit for AI-dr
```

Using the GUI

In order to access the user interface locally, either by Podman/Docker or Python code, run the following commands respectively.

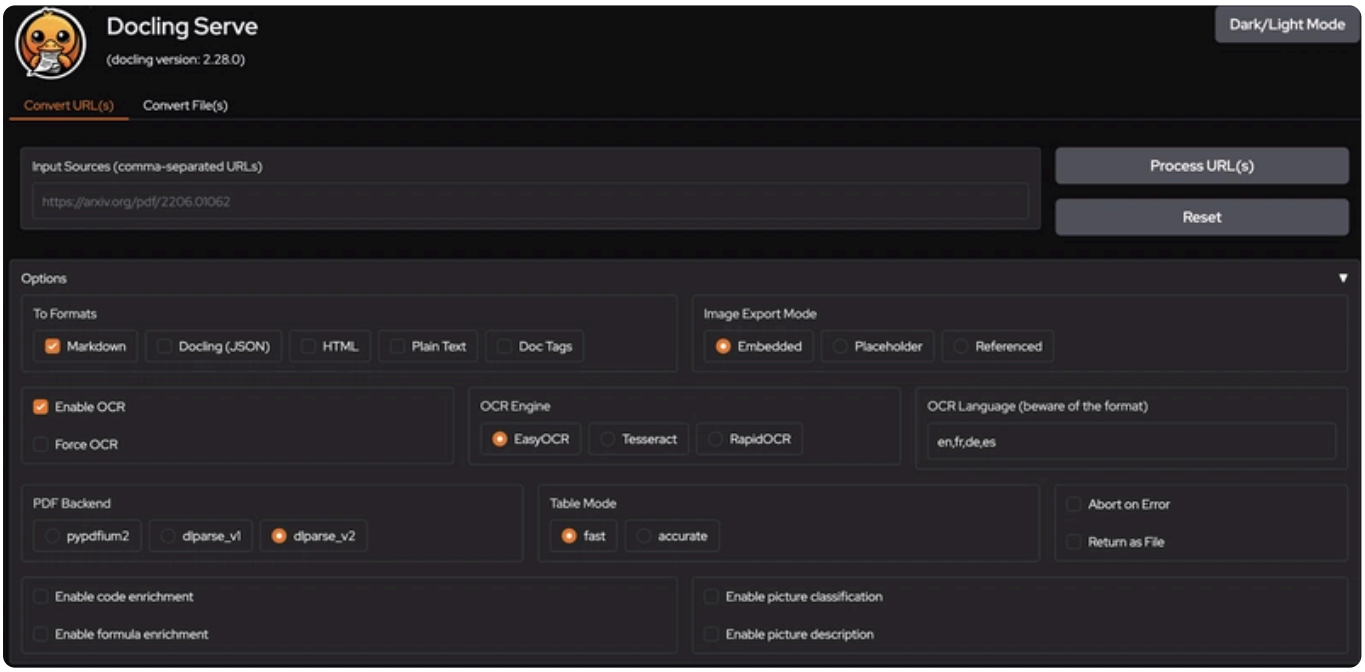
- Access to the UI through the container based image.

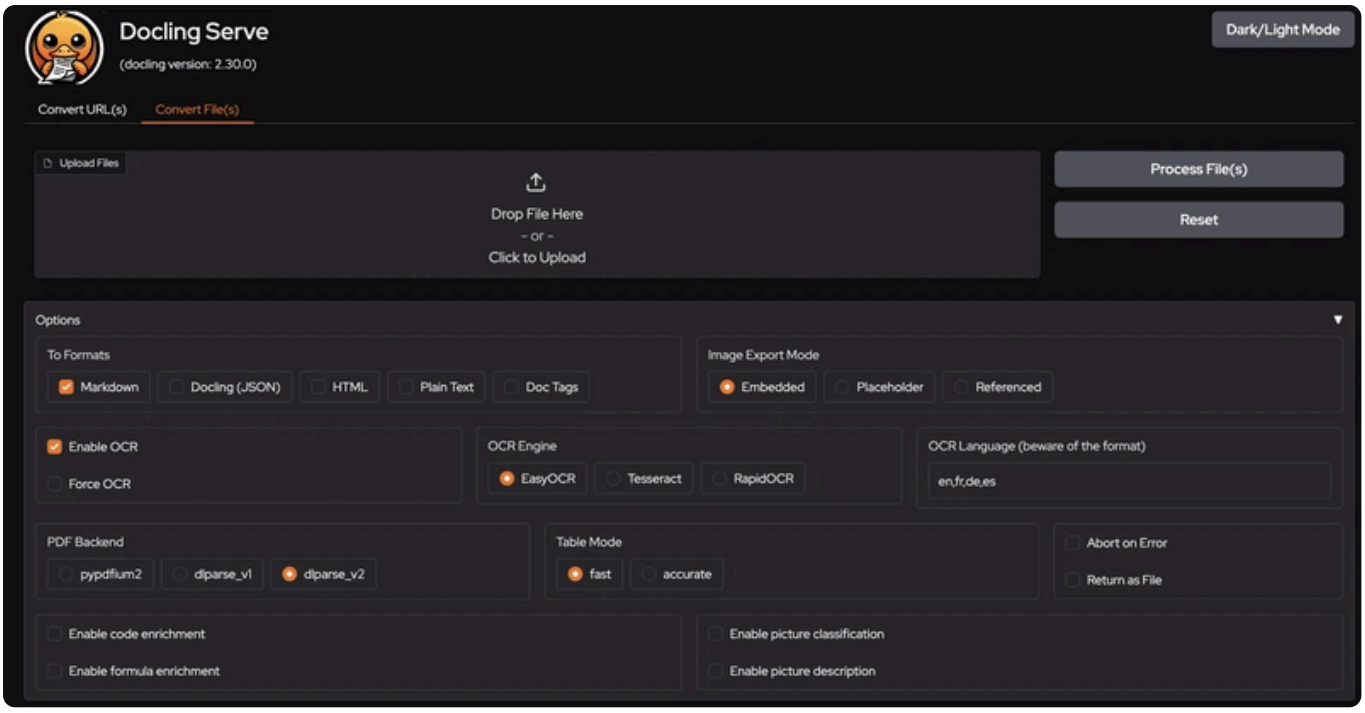
```
# for podman
podman run -p 5001:5001 -e DOCLING_SERVE_ENABLE_UI=true quay.io/docling-proje
# for docker
docker run -p 5001:5001 -e DOCLING_SERVE_ENABLE_UI=true quay.io/docling-proje
```

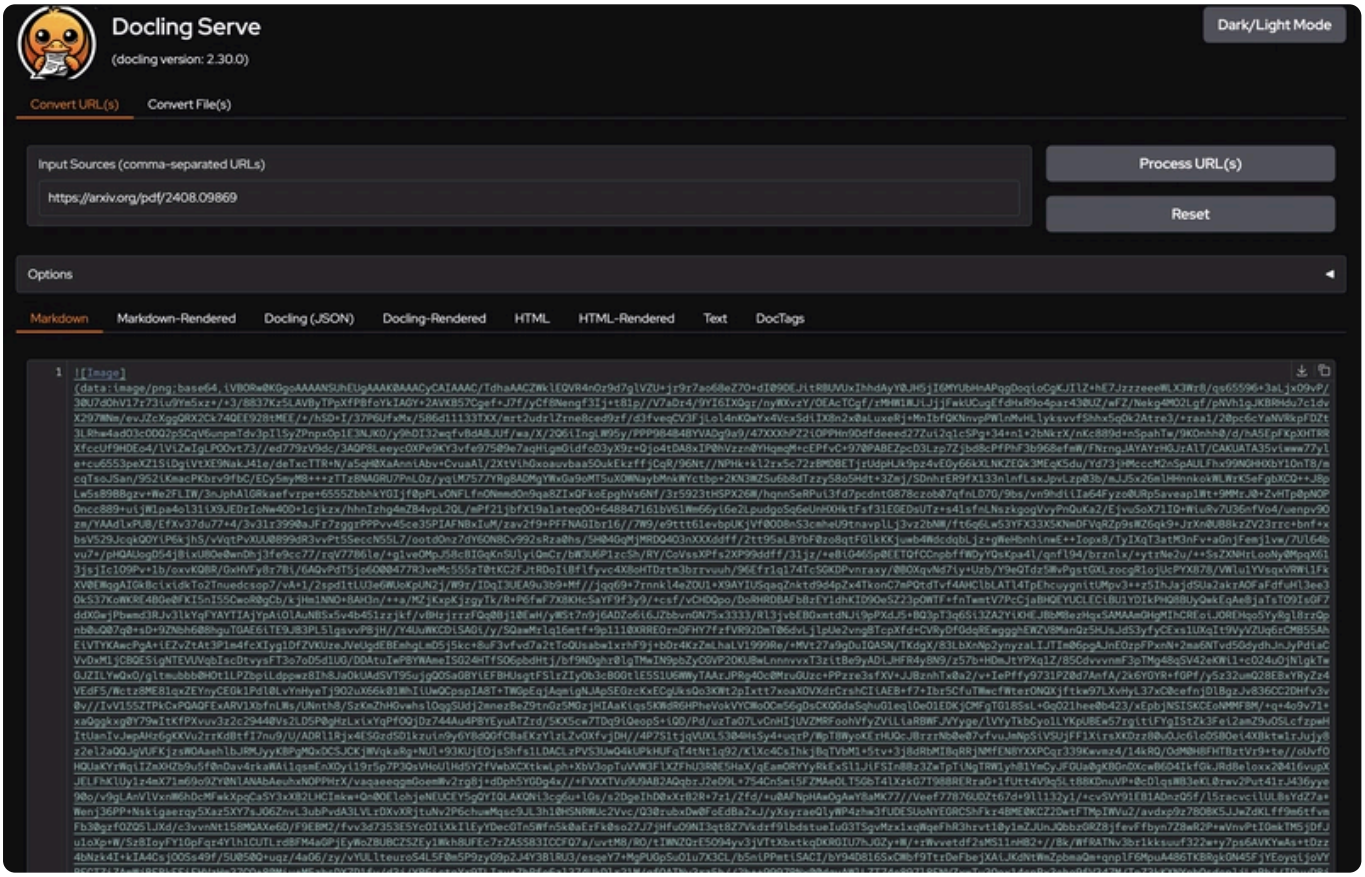
- Access to the UI through the code.

```
pip install "docling-serve[ui]"
docling-serve run --enable-ui

####

~/Devs/doclingserve-test docling-serve run --enable-ui
Starting production server 🚀

Server started at http://0.0.0.0:5001
Documentation at http://0.0.0.0:5001/docs
UI at http://0.0.0.0:5001/ui
```

- Now we're going to use the functionalities through the UI.




Docling Serve
(docling version: 2.30.0)
Dark/Light Mode
Convert URL(s)
Convert File(s)
Upload Files
Drop File Here
- or -
Click to Upload
Process File(s)
Reset
Options
To Formats
Markdown
Docling (JSON)
HTML
Plain Text
Doc Tags
Image Export Mode
Embedded
Placeholder
Referenced
Enable OCR
Force OCR
OCR Engine
EasyOCR
Tesseract
RapidOCR
OCR Language (beware of the format)
en,fr,de,es
PDF Backend
pypdfium2
dlparse_v1
dlparse_v2
Table Mode
fast
accurate
Abort on Error
Return as File
Enable code enrichment
Enable formula enrichment
Enable picture classification
Enable picture description


Docling Serve
(docling version: 2.30.0)
Dark/Light Mode
Convert URL(s)
Convert File(s)
Input Sources (comma-separated URLs)
https://arxiv.org/pdf/2408.09869
Process URL(s)
Reset
Options
Markdown
Markdown-Rendered
Docling (JSON)
Docling-Rendered
HTML
HTML-Rendered
Text
DocTags

```
3  ## Docling Technical Report
4
5  Version 1.0
6
7  Christoph Auer Maksym Lysak Ahmed Nassar Michele Dolfi Nikolaos Livathinos Panos Vagenas Cesar Berrospi Ramis Matteo Omenetti Fabian Lindlbauer Kasper Dinkla Lokesh Mishra Yusik Kim Shubham
   Gupta Rafael Teixeira de Lima Valery Weber Lucas Morin Ingmar Meijer Viktor Kuropiatnyk Peter W. J. Staar
8
9  AI4K Group, IBM Research R¨ uschlikon, Switzerland
10
11 ## Abstract
12
13 This technical report introduces Docling , an easy to use, self-contained, MITlicensed open-source package for PDF document conversion. It is powered by state-of-the-art specialized AI models
   for layout analysis (DocLayNet) and table structure recognition (TableFormer), and runs efficiently on commodity hardware in a small resource budget. The code interface allows for easy
   extensibility and addition of new features and models.
14
15 ## 1 Introduction
16
17 Converting PDF documents back into a machine-processable format has been a major challenge for decades due to their huge variability in formats, weak standardization and printing-optimized
   characteristic, which discards most structural features and metadata. With the advent of LLMs and popular application patterns such as retrieval-augmented generation (RAG), leveraging the rich
   content embedded in PDFs has become ever more relevant. In the past decade, several powerful document understanding solutions have emerged on the market, most of which are commercial software,
   cloud offerings [3] and most recently, multi-modal vision-language models. As of today, only a handful of open-source tools cover PDF conversion, leaving a significant feature and quality gap
   to proprietary solutions.
18
19 With Docling , we open-source a very capable and efficient document conversion tool which builds on the powerful, specialized AI models and datasets for layout analysis and table structure
   recognition we developed and presented in the recent past [12, 13, 9]. Docling is designed as a simple, self-contained python library with permissive license, running entirely locally on
   commodity hardware. Its code architecture allows for easy extensibility and addition of new features and models.
20
21 Here is what Docling delivers today:
22
23 - · Converts PDF documents to JSON or Markdown format, stable and lightning fast
24 - · Understands detailed page layout, reading order, locates figures and recovers table structures
25 - · Extracts metadata from the document, such as title, authors, references and language
26 - · Optionally applies OCR, e.g. for scanned PDFs
27 - · Can be configured to be optimal for batch-mode (i.e high throughput, low time-to-solution) or interactive mode (compromise on efficiency, low time-to-solution)
28 - · Can leverage different accelerators (GPU, MPS, etc).
29
30 ## 2 Getting Started
31
32 To use Docling, you can simply install the docling package from PyPI. Documentation and examples are available in our GitHub repository at github.com/DS4SD/docling. All required model assets 1
   are downloaded to a local huggingface datasets cache on first use, unless you choose to pre-install the model assets in advance.
33
34 Docling provides an easy code interface to convert PDF documents from file system, URLs or binary streams, and retrieve the output in either JSON or Markdown format. For convenience, separate
   methods are offered to convert single documents or batches of documents. A basic usage example is illustrated below. Further examples are available in the Doclign code repository.
35
36 ```
37 from docling.document_converter import DocumentConverter Large
38 ```
39
40 ```
41 source = "https://arxiv.org/pdf/2206.01062" # PDF path or URL converter = DocumentConverter() result = converter.convert_single(source) print(result.render_as_markdown()) # output: "##
   DocLayNet: A Human -Annotated Dataset for Document -Layout Analysis [...]"
42 ```
43
```

```
219 Figure 1: Four examples of complex page layouts across different document categories
220
221 ## KEYWORDS
222
223 PDF document conversion, layout segmentation, object-detection, data set, Machine Learning
224
225 ## ACMReference Format:
226
227 Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. 2022. DocLayNet: A Large Human-Annotated Dataset for DocumentLayout Analysis. In Proceedings of the 28th ACM
    SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22), August 14-18, 2022, Washington, DC, USA. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/ 3534678.3539043
228
229 1 INTRODUCTION
230
231 Despite the substantial improvements achieved with machine-learning (ML) approaches and deep neural networks in recent years, document conversion remains a challenging problem, as demonstrated
    by the numerous public competitions held on this topic [1-4]. The challenge originates from the huge variability in PDF documents regarding layout, language and formats (scanned, programmatic
    or a combination of both). Engineering a single ML model that can be applied on all types of documents and provides high-quality layout segmentation remains to this day extremely challenging
    [5]. To highlight the variability in document layouts, we show a few example documents from the DocLayNet dataset in Figure 1. Figure 2: Title page of the DocLayNet paper
    (arxiv.org/pdf/2206.01062) - left PDF, right rendered Markdown. If recognized, metadata such as authors are appearing first under the title. Text content inside figures is currently dropped,
    the caption is retained and linked to the figure in the JSON representation (not shown).
232
233 KDD '22, August 14-18, 2022, Washington, DC, USA Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar
234
235 Table 2: Prediction performance (mAP@0.5-0.95) of object detection networks on DocLayNet test set. The MRCNN (Mask R-CNN) and FRCNN (Faster R-CNN) models with ResNet-50 or ResNet-101 backbone
    were trained based on the network architectures from the detectron2 model zoo (Mask R-CNN R50, R101-FPN 3x, Faster R-CNN R101-FPN 3x), with default configurations. The YOLO implementation
    utilized was YOLOv5x6 [13]. All models were initialised using pre-trained weights from the COCO 2017 dataset.
236
237 |             | human   | MRCNN   | MRCNN   | FRCNN   | YOLO   |
238 |-------------|---------|---------|---------|---------|--------|
239 |             | human   | R50     | R101    | R101    | v5x6   |
240 | Caption     | 84-89   | 68.4    | 71.5    | 70.1    | 77.7   |
241 | Footnote    | 83-91   | 70.9    |         | 73.7    | 77.2   |
242 | Formula     | 83-85   | 60.1    |         | 63.5    | 66.2   |
243 | List-item   | 87-88   | 81.2    | 80.8    | 81.0    | 86.2   |
244 | Page-footer | 93-94   | 61.6    | 59.3    | 58.9    | 61.1   |
245 | Page-header | 85-89   | 71.9    | 70.0    | 72.0    | 67.9   |
246 | Picture     | 69-71   | 71.7    | 72.7    |         |        |
247 |             | 83-84   | 67.6    | 69.3    | 68.4    | 74.6   |
248 | Table       | 77-81   | 82.2    | 82.9    | 82.2    | 86.3   |
249 | Text        | 84-86   |         | 85.8    | 85.4    |        |
250 |             |         | 76.7    | 80.4    | 79.9    | 88.1   |
251 | Title       |         |         |         |         | 82.7   |
252 | All         |         | 72.4    | 73.5    | 73.4    | 76.8   |
253
254 to avoid this at any cost in order to have clear, unbiased baseline numbers for human document-layout annotation. Third, we introduced the feature of snapping boxes around text segments to
    obtain a pixel-accurate annotation and again reduce time and effort. The CCS annotation tool automatically shrinks every user-drawn box to the minimum bounding-box around the enclosed text-
    cells for all purely text-based segments, which excludes only Table and Picture . For the latter, we instructed annotation staff to minimise inclusion of surrounding whitespace while including
    all graphical lines. A downside of snapping boxes to enclosed text cells is that some wrongly parsed PDF pages cannot be annotated correctly and need to be skipped. Fourth, we established a
    way to flag pages as rejected for cases where no valid annotation according to the label guidelines could be achieved. Example cases for this would be PDF pages that render incorrectly or
    contain layouts that are impossible to capture with non-overlapping rectangles. Such rejected pages are not contained in the final dataset. With all these measures in place, experienced
    annotation staff managed to annotate a single page in a typical timeframe of 20s to 60s, depending on its complexity.
255
256 ## 5 EXPERIMENTS
```

Deployment on a cluster (OpenShift / Kubernetes)

In order to deploy the API server on a cluster based environment, do the following ⬇️.

```
kubectl apply -f docs/deploy-examples/docling-serve-oauth.yaml
```

```
# Retrieve the endpoint on an OpenShift server
DOCLING_NAME=docling-serve
DOCLING_ROUTE="https://$(oc get routes ${DOCLING_NAME} --template={{.spec.hos

# Retrieve the authentication token
OCP_AUTH_TOKEN=$(oc whoami --show-token)

# Make a test query
curl -X 'POST' \
  "${DOCLING_ROUTE}/v1alpha/convert/source/async" \
  -H "Authorization: Bearer ${OCP_AUTH_TOKEN}" \
  -H "accept: application/json" \
  -H "Content-Type: application/json" \
  -d '{
    "http_sources": [{"url": "https://arxiv.org/pdf/2501.17887"}]
  }'


# docling-serve-oauth.yaml
# This example deployment configures Docling Serve with a OAuth-Proxy sidecar

---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: docling-serve
  labels:
    app: docling-serve
  annotations:
    serviceaccounts.openshift.io/oauth-redirectreference.primary: '{"kind":"(
---
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: docling-serve-oauth
  labels:
    app: docling-serve
    component: docling-serve-api
rules:
  - verbs:
      - create
    apiGroups:
      - authorization.k8s.io
    resources:
      - subjectaccessreviews
  - verbs:
      - create
    apiGroups:
```

```yaml
      - authentication.k8s.io
    resources:
      - tokenreviews
---
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: docling-serve-oauth
  labels:
    app: docling-serve
    component: docling-serve-api
subjects:
  - kind: ServiceAccount
    name: docling-serve
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: docling-serve-oauth
---
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: docling-serve
  labels:
    app: docling-serve
    component: docling-serve-api
spec:
  to:
    kind: Service
    name: docling-serve
  port:
    targetPort: oauth
  tls:
    termination: Reencrypt
---
apiVersion: v1
kind: Service
metadata:
  name: docling-serve
  labels:
    app: docling-serve
    component: docling-serve-api
  annotations:
    service.alpha.openshift.io/serving-cert-secret-name: docling-serve-tls
spec:
  ports:
  - name: oauth
    port: 8443
```

```yaml
    targetPort: oauth
  - name: http
    port: 5001
    targetPort: http
  selector:
    app: docling-serve
    component: docling-serve-api
---
kind: Deployment
apiVersion: apps/v1
metadata:
  name: docling-serve
  labels:
    app: docling-serve
    component: docling-serve-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: docling-serve
      component: docling-serve-api
  template:
    metadata:
      labels:
        app: docling-serve
        component: docling-serve-api
    spec:
      restartPolicy: Always
      serviceAccountName: docling-serve
      containers:
        - name: api
          resources:
            limits:
              cpu: 500m
              memory: 2Gi
            requests:
              cpu: 250m
              memory: 1Gi
          readinessProbe:
            httpGet:
              path: /health
              port: http
              scheme: HTTPS
            initialDelaySeconds: 10
            timeoutSeconds: 2
            periodSeconds: 5
            successThreshold: 1
            failureThreshold: 3
```

```yaml
      livenessProbe:
        httpGet:
          path: /health
          port: http
          scheme: HTTPS
        initialDelaySeconds: 3
        timeoutSeconds: 2
        periodSeconds: 5
        successThreshold: 1
        failureThreshold: 3
      env:
        - name: DOCLING_SERVE_ENABLE_UI
          value: 'true'
        - name: UVICORN_SSL_CERTFILE
          value: '/etc/tls/private/tls.crt'
        - name: UVICORN_SSL_KEYFILE
          value: '/etc/tls/private/tls.key'
      ports:
        - name: http
          containerPort: 5001
          protocol: TCP
      volumeMounts:
        - name: proxy-tls
          mountPath: /etc/tls/private
      imagePullPolicy: Always
      image: 'ghcr.io/docling-project/docling-serve'
    - name: oauth-proxy
      resources:
        limits:
          cpu: 100m
          memory: 256Mi
        requests:
          cpu: 100m
          memory: 256Mi
      readinessProbe:
        httpGet:
          path: /oauth/healthz
          port: oauth
          scheme: HTTPS
        initialDelaySeconds: 5
        timeoutSeconds: 1
        periodSeconds: 5
        successThreshold: 1
        failureThreshold: 3
      livenessProbe:
        httpGet:
          path: /oauth/healthz
          port: oauth
```

```
            scheme: HTTPS
          initialDelaySeconds: 30
          timeoutSeconds: 1
          periodSeconds: 5
          successThreshold: 1
          failureThreshold: 3
        ports:
          - name: oauth
            containerPort: 8443
            protocol: TCP
        imagePullPolicy: IfNotPresent
        volumeMounts:
          - name: proxy-tls
            mountPath: /etc/tls/private
        env:
          - name: NAMESPACE
            valueFrom:
              fieldRef:
                fieldPath: metadata.namespace
        image: 'registry.redhat.io/openshift4/ose-oauth-proxy:v4.13'
        args:
          - '--https-address=:8443'
          - '--provider=openshift'
          - '--openshift-service-account=docling-serve'
          - '--upstream=https://docling-serve.$(NAMESPACE).svc.cluster.loca
          - '--upstream-ca=/var/run/secrets/kubernetes.io/serviceaccount/se
          - '--tls-cert=/etc/tls/private/tls.crt'
          - '--tls-key=/etc/tls/private/tls.key'
          - '--cookie-secret=SECRET'
          - '--openshift-delegate-urls={"/": {"group":"route.openshift.io",
          - '--openshift-sar={"namespace":"$(NAMESPACE)","resource":"routes
          - '--skip-auth-regex=''(^/health|^/docs)'''
      volumes:
        - name: proxy-tls
          secret:
            secretName: docling-serve-tls
            defaultMode: 420
```

Conclusion

In essence, Docling-serve effectively bridges the gap between user interaction and programmatic access. By providing both a user-friendly interface and a robust API server, it empowers end users to seamlessly harness its full spectrum of functionalities, whether through intuitive visual tools or direct API calls, offering unparalleled flexibility and accessibility.

Links

- Docling-serve: https://github.com/docling-project/docling-serve
- Readme: https://github.com/docling-project/docling-serve/blob/main/README.md
- Cluster deployment: https://github.com/docling-project/docling-serve/blob/main/docs/deployment.md
- Usage: https://github.com/docling-project/docling-serve/blob/main/docs/usage.md

The DEV Team PROMOTED

Your mandate is to build the most impressive system or process using Notion MCP.

Running through March 29, the Notion MCP Challenge welcomes you to centralize your workflow with AI-powered docs, projects, and notes. Scale side hustles or empires with "human-in-the-loop" systems that run globally without ever hitting snooze!

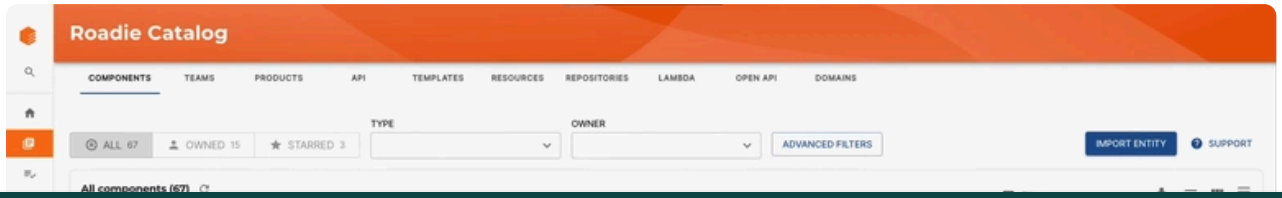
See more 👀

Top comments (2)

Some comments may only be visible to logged-in visitors. Sign in to view all comments.

Code of Conduct • Report abuse

Draft.dev PROMOTED



Alain Airom

Senior Engineer - Partner Technical Specialist - 30+ years experience in the IT industry. Always learning!

LOCATION
France

WORK
Architect at IBM

JOINED
13 Jul 2020

More from Alain Airom

See the comparison

Bob Strikes Again: ‘PageIndex’ Test and Implementation
#bob #rag #pageindex

Docling-Agent is Coming: Bob’s Blueprint for Next-Gen Automation
#bob #docling #agents #doclingagent

Book Review: “Mastering NLP From Foundations to Agents - 2nd Edition”
#bookreview #rag #agents #nlp

Draft.dev PROMOTED

How to Automate Document Creation with a Document Generation API

Still generating invoices, contracts, or reports manually? Learn how a document generation API turns templates + JSON into fully formatted PDFs at scale.

Read more